BIASHARA MALL

The agent, and the rails it runs on

How Biashara Mall becomes an agent with tools and a decision layer — and what has to exist before that is safe. Written 2026-08-18, in response to a proposed structure. Reviews that proposal honestly, then restructures it.

The verdict, up front

The layering instinct is right. Frontend → API → runtime → tools → storage is the correct shape, and two pieces already exist: `ScraperEngine` and `Drafter` are `Protocol`s, which is exactly the seam a tool interface needs. Adding a third tool is a new class, not a new pattern.

Three things are wrong, and one of them is serious.

	PROPOSED	VERDICT
Next.js	frontend	❌ Not applicable — the frontend is Jinja2 + HTMX and is built
RAG + pgvector/Chroma	retrieval	❌ Nothing to retrieve semantically. Contradicts a standing rule
Agent API / User API	two APIs	⚠️ One API, two callers. Tools must not speak HTTP to us
Job queue	*absent*	🔴 **The serious gap.** Every tool is slow and costs money
Spend ledger	*absent*	🔴 An agent that loops can spend without limit
Decision layer	named, not placed	⚠️ Wanted, but the diagram goes Runtime → Tools directly
Memory	one box	⚠️ Three different things wearing one name
Object storage	bottom	✅ Correct, and `services/media.py` already stores relative paths for it

1. Next.js — already answered

The frontend is server-rendered Jinja2 with HTMX, deployed as one FastAPI process. That was decided for a reason that has not changed: the buyer storefront opens inside the WhatsApp in-app browser on a cheap Android over expensive data, and a JS framework is a tax paid by every visitor.

Nothing in the agent plan needs a SPA. An agent that takes 40 seconds to draft a caption is better served by a job row and a polling partial than by a websocket — and HTMX does polling in one attribute.

2. No RAG. Still no RAG.

`CLAUDE.md` says it plainly:

> **No RAG.** Every lookup is by known key. Don't bring a vector database to a `WHERE id = ?` problem.

That rule survives the pivot to a content engine. Walk the actual questions the agent has to answer:

THE QUESTION	THE RETRIEVAL
"How am I doing?"	`WHERE social_account_id = ? ORDER BY captured_at`
"Which of my posts worked?"	`ORDER BY views DESC` — a known key and a sort
"Who else is in my niche?"	An Apify keyword search. An API call, not retrieval
"What hooks work for mitumba?"	`WHERE category = ? ORDER BY engagement_ratio DESC`
"Write a caption like my best one"	Put the best one in the prompt. Known key

None of these need embeddings. Every one is a filter and a sort over rows we own. A vector index would return *approximately* what a `WHERE` clause returns exactly, at the cost of an embedding bill on every write and a second thing to keep in sync.

When it would genuinely earn its place

One question does not reduce to a key: "*find posts that mean something similar to this idea, across a corpus too large to scan.*" That needs a competitor corpus in the tens of thousands, which is Phase 3 at the earliest.

When that day comes it is pgvector, not Chroma. We already run Postgres, and "one database per application" is a rule. Chroma would be a second datastore, a second backup story and a second thing to be down — for a feature Postgres does natively.

The retrofit is one column and one index. **Nothing in this document needs to change to allow it**, which is the real test of whether deferring is safe.

3. Not two APIs. One service layer, two callers.

The proposal splits FastAPI into an Agent API and a User API. That looks tidy and costs more than it looks.

If tools call our own HTTP endpoints we get a second authentication surface to secure, a network hop to ourselves inside every tool call, and two paths that can disagree about a business rule.

The layering rule already answers this. Routes are thin and delegate to `services/`. A tool is simply another caller:

One place holds each rule, and it is tested once. A tool that needs something a route already does calls the same function the route calls.

---

4. The serious gap: there is no queue

Every external call in this product is slow, costs money, or both.

TOOL	TYPICAL DURATION	COST
Apify profile scrape	30 s – 3 min	per post
Gemini video tier	10 – 60 s	per video
Canva render	seconds – minutes	per design
CapCut export	minutes	CPU

Running these inside an HTTP request is a bug, not an optimisation. The request times out, the proxy returns 502, the seller presses the button again — and the second press pays for the first one's work a second time. The user sees a broken product and we see the bill.

`ScrapeJob` already exists and already does the right things: it records status, timing, counts and the error, and it is written *before* the failure escapes so the dashboard can explain itself. **Generalise it.**

```
route/agent ──▶ enqueue Job (queued) ──▶ returns immediately
                       │
                worker picks it up
                       │
                running ──▶ succeeded | failed
                       │
                HTMX polls /jobs/{id} and swaps in the result
```

One extra Railway process runs the worker. No Celery, no Redis, no broker: a `SELECT … FOR UPDATE SKIP LOCKED` against Postgres is a queue, and it is a queue we already operate, back up and monitor.

> **Why not a broker.** Redis is a second datastore for a workload measured in hundreds of jobs a day. Postgres does this correctly at our volume, and "fewer moving parts" is worth more than throughput we will not use.

5. The other serious gap: nothing counts the money

An agent's defining feature is that it decides how many times to call a tool. That is the whole value, and it is also how an agent produces a four-figure bill overnight.

`services/ingestion.py` already contains a hand-rolled instance of the fix — the once-per-day cooldown, checked *before* the paid call, with a test asserting that ordering. That instinct needs to become infrastructure:

A `tool_run` ledger. Every tool call, recorded: which tool, which seller, the arguments, the outcome, the duration, and **the estimated cost**. It pays for itself three times over — cost attribution per seller, debugging what the agent actually did, and idempotency, because a call already made and paid for can be returned from the ledger instead of made again.

A budget, enforced before the call. Per seller, per day. Checked in the decision layer, never in the tool — a tool that polices itself is a tool that can be bypassed by adding another tool.

---

6. The decision layer

This is the piece the proposal names but does not place. It belongs between the runtime and the tools, and it is **ordinary deterministic code** — no model involved in deciding whether the model may act.

That is `CLAUDE.md`'s first architecture rule restated:

> **The agent proposes, code disposes.** The LLM never decides price, stock, payment status or consent. It drafts; deterministic code transacts.

Every tool declares what it is, and the decision layer reads that declaration:

```
class ToolSpec:
    name: str
    reads: bool          # does it only look?
    spends: bool         # does it cost money?
    writes: bool         # does it change our data?
    publishes: bool      # is it visible outside Biashara Mall?
    max_per_day: int
```

Four gates, in order, and the order matters:

1. **Permission** — is this tool allowed for this seller's plan and connections?
2. **Rate** — has `max_per_day` been reached? (the cooldown, generalised)
3. **Budget** — would this call exceed today's spend?
4. **Approval** — does the result need a human before it takes effect?

Gate 4 is where `publishes: true` **always stops.** Nothing reaches TikTok, Instagram, YouTube or a buyer's WhatsApp without a person pressing a button. This is not a policy we can relax later for convenience — it is the promise the signup page makes in writing:

> **User approval first.** Publish to TikTok, Instagram or YouTube when ready.

A promise printed on the signup page has to be enforced somewhere in the code. This is that somewhere.

Read tools are free

Tools that only read our own database — "what were my top posts", "what is my average" — pass every gate trivially and can be called as often as the agent likes. **The expensive gates only apply to expensive calls.** This is what keeps an agent loop from being either dangerous or uselessly slow.

---

7. Three things called "memory"

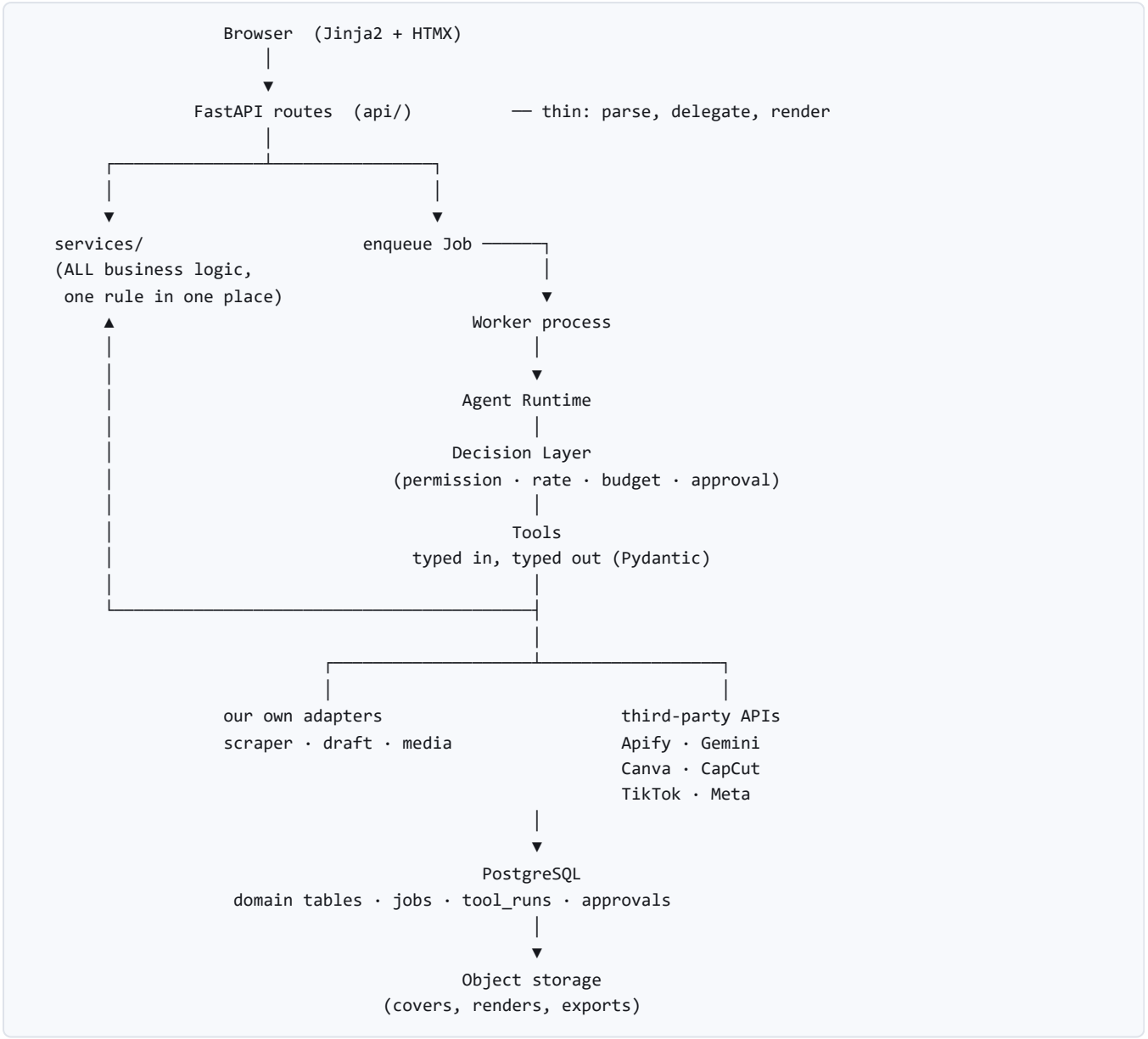
The proposal has one box. It is three, and only one of them is memory in the LLM sense.

WHAT	WHERE	NEEDED
Run state — this task, mid-flight	the `Job` row	With the queue
Durable facts — what we drafted, published, what performed	ordinary tables	Phase 0
Conversation — what the seller said last turn	a `messages` table	P5, Soko AI

Only the third is conversational memory, and nothing before Soko AI needs it. The first two are rows, and calling them memory invites someone to reach for a vector store to solve them.

---

8. The corrected structure



Three things to read out of that diagram:

- **Everything bottoms out in `services/`.** Routes and tools are both callers. No rule lives in two places.
- **The decision layer is not optional and not bypassable.** A tool is reached through it or not at all.
- **pgvector is absent, and its absence changes nothing structurally.** Adding it later is a column on a table that does not exist yet.

---

9. What to build, and when

The useful discovery: **most of the agent's foundations are things Phase 0 needs anyway.** This is not a detour.

Now — Phase 0, already planned

	WHY IT IS ALSO AGENT WORK
`Post` + `PostMetricSnapshot`	The corpus the agent reasons over. **History cannot be backfilled**
Analytics sync	The first thing worth running as a job

Next — the rails, before any agent

	WHY NOW RATHER THAN LATER
Generalise `ScrapeJob` → `Job` + worker	Analytics sync already wants it. Retrofitting async onto sync code touches every caller
`tool_run` ledger	Apify and Gemini cost money *today*. Un-ledgered spend cannot be reconstructed afterwards
`ToolSpec` + the four gates	Cheap while there are two tools. Expensive once there are eight

Then — the agent itself

Only once the rails hold. `CLAUDE.md` again:

> **Rails before agent.** Payment paths are built and tested before an agent may call them.

The same logic covers spend: **the budget is built and tested before an agent may spend.**

The order, and why

```
Post + snapshots ──▶ Job + worker ──▶ ledger + gates ──▶ agent runtime
     │                   │                 │                  │
corpus to          work that takes    money that is      decides how
reason over        longer than a      already being       often to act
                   request            spent
```

Each step is independently useful. Stop after any one of them and the product is better than it was — which is the test of whether a plan is real or is scaffolding for a plan.

---

10. What this does not answer

Open, and worth deciding before the runtime is written:

1. **Which onboarding flow is real** — the mockup's *create → make content → review & publish*, or Phase 0's *connect → sync → analytics*. The signup page currently promises the first, and steps 2 and 3 do not exist. **The agent's first job is whichever one wins.**
2. **What the agent is actually for, first.** "Draft three hooks from my last ten posts" is a different system from "answer a buyer's question in WhatsApp". Both are on the roadmap; the first one built shapes the runtime.
3. **Which model, and where the seam is.** `agent/draft.py` is already the only file that knows we use Gemini. A second agent must not add a second place that knows.

4. **Budget per seller, in shillings.** The gates are worthless without a number. It should come from the pricing, not from a guess.
